

GUIA DE ESTUDOS: COUNTING SORT E RADIX SORT (LSD & MSD)

Instituição: Instituto Federal Sul-rio-grandense (IFSUL) - Câmpus Passo Fundo

Curso: Bacharelado em Ciência da Computação

Disciplina: Estrutura de Dados III

Professor: Prof. Leonardo Deliyannis Constantin

1. INTRODUÇÃO E LIMITES DA ORDENAÇÃO

O Limite Teórico por Comparação: $\Omega(n \log n)$

Na maioria dos algoritmos clássicos estudados (Quick Sort, Merge Sort, Heap Sort, Insertion Sort), a ordenação é executada por meio de comparações sequenciais entre pares de chaves ($A[i] \leq A[j]$).

Formalmente, qualquer algoritmo baseado em comparação pode ser modelado como uma **Árvore de Decisão Binária**:

- Cada nó interno representa uma comparação entre dois elementos.
- Cada folha representa uma das possíveis permutações das chaves de entrada.
- Para um vetor de tamanho n , existem $n!$ permutações possíveis.
- Para distinguir entre $n!$ folhas, uma árvore binária precisa de altura h :

$$2^h \geq n! \implies h \geq \log_2(n!) \approx n \log_2 n - n \log_2 e = \Omega(n \log n)$$

Portanto, **nenhum algoritmo baseado em comparação consegue ter melhor desempenho no pior caso do que $\Omega(n \log n)$** .

Quebrando a Barreira: Ordenação em Tempo Linear

Algoritmos como o **Counting Sort** e o **Radix Sort** não realizam comparações diretas de grandeza entre os itens. Eles utilizam a própria representação e propriedades matemáticas das chaves (valores inteiros discretos, representação posicional em bases numéricas ou strings)

como índices diretos de acesso em memória, possibilitando tempo de execução linear $O(n)$ sob condições favoráveis de entrada.

2. COUNTING SORT (ORDENAÇÃO POR CONTAGEM)

O Counting Sort assume que cada elemento de entrada pertence a um conjunto discreto de inteiros no intervalo $[0, k]$.

Etapas do Algoritmo

1. Contagem de Ocorrências:

Cria-se um vetor auxiliar C com tamanho $k + 1$, inicializado com zeros. Para cada chave presente no vetor original A , incrementa-se $C[A[i]]$.

2. Somas Acumuladas (Prefixadas):

Calcula-se a soma cumulativa em C :

$$C[i] = C[i] + C[i - 1]$$

Agora, $C[i]$ armazena quantos elementos no conjunto são menores ou iguais a i . Essa informação determina o índice exato onde a chave i deve ser posicionada no vetor final.

3. Distribuição Estável no Vetor de Saída:

Percorre-se o vetor A obrigatoriamente da direita para a esquerda (do índice $n - 1$ até 0):

- O elemento $A[i]$ é alocado na posição $C[A[i]] - 1$ do vetor de saída B .
- O contador $C[A[i]]$ é decrementado em 1.
- *Nota pedagógica:* Essa varredura de trás para frente é o que garante a **estabilidade** do algoritmo (elementos iguais mantêm a ordem original).

Análise de Complexidade e Propriedades

- Tempo no Melhor Caso: $\Theta(n + k)$
- Tempo no Médio Caso: $\Theta(n + k)$
- Tempo no Pior Caso: $\Theta(n + k)$
- Espaço Auxiliar: $\Theta(n + k)$ (necessita do vetor auxiliar C de tamanho $k + 1$ e do vetor de saída B de tamanho n).
- In-Place? Não.

- **Estável?** Sim.
- **Quando usar?** Quando $k = O(n)$ (o valor máximo das chaves não cresce muito mais rápido que a quantidade de elementos).
- **Quando evitar?** Quando $k \gg n$ (ex: ordenar 10 inteiros cujos valores podem chegar a 1.000.000, gerando desperdício massivo de memória e processamento inútil no vetor C).

3. RADIX SORT: LSD VS. MSD

O Radix Sort soluciona o problema de chaves com valores altos dividindo os números em d dígitos em uma determinada base b .

3.1 Radix Sort LSD (Least Significant Digit)

- **Princípio:** Inicia a ordenação pelo dígito **menos significativo** (unidade) e avança progressivamente para a esquerda (dezenas, centenas, etc.).
- **Regra Mandatória:** A sub-rotina de ordenação usada para ordenar cada dígito **PRECISA SER ESTÁVEL**. Se a ordenação do dígito não for estável, a ordenação dos dígitos inferiores anteriores é desfeita.
- **Complexidade:**
 - Tempo: $\Theta(d \cdot (n + b))$ para melhor, pior e médio caso.
 - Espaço: $O(n + b)$.
- **Vantagem:** Muito simples de implementar iterativamente, ideal para números inteiros ou chaves de tamanho fixo.

3.2 Radix Sort MSD (Most Significant Digit)

- **Princípio:** Inicia a ordenação pelo dígito **mais significativo** e avança para a direita utilizando partição recursiva (Divisão e Conquista).
- **Funcionamento:** Agrupa os elementos em baldes (*buckets*) baseados no primeiro dígito. Em seguida, chama o algoritmo recursivamente para cada balde considerando o próximo dígito.
- **Complexidade:**
 - Tempo de Melhor Caso: $\Omega(n)$ (quando os elementos já se diferenciam fortemente nos primeiros dígitos).
 - Pior Caso: $O(d \cdot (n + b))$.
 - Espaço: $O(n + d \cdot b)$ (devido à pilha de chamadas de recursão e criação de baldes).
- **Vantagem:** Excelente para ordenar **strings** e sequências de comprimentos variáveis

○

4. MATERIAIS DE APOIO RECOMENDADOS

- https://www.youtube.com/watch?v=XiuSW_mEn7g
- <https://www.youtube.com/watch?v=GpkRTKKkTqs>
- <https://github.com/paladini/awesome-poscomp>
- <https://github.com/geacc/enade>